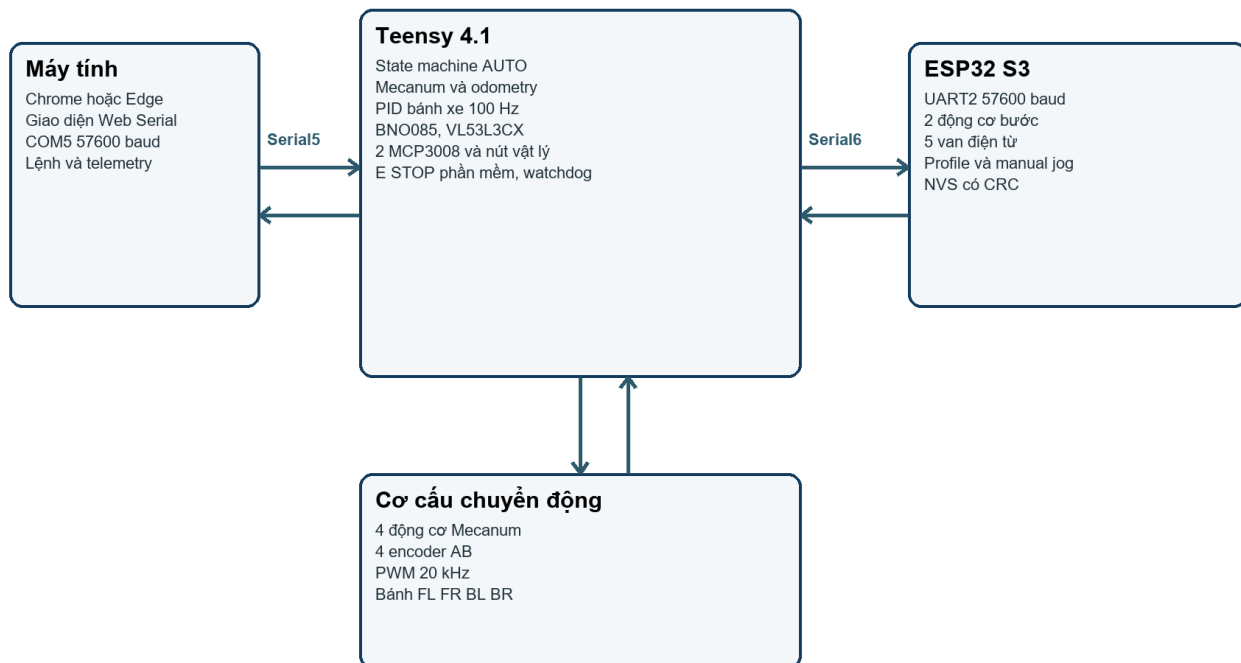


Báo cáo kỹ thuật hệ thống điều khiển robot ROBOCON 2026

Teensy 4.1 ESP32 S3 Giao diện Web Serial

Kiến trúc điều khiển và đường dữ liệu



Hình 1 Kiến trúc tổng thể của hệ thống

Thông tin	Giá trị
Mục đích	Giải thích toàn bộ source hiện tại để vận hành, hiệu chỉnh, bảo trì và kiểm soát thay đổi
Phạm vi	Firmware Teensy, firmware ESP32 cơ cấu thật, firmware ESP32 giả lập, giao diện telemetry và các công cụ chẩn đoán
Ngày chụp source	08 tháng 09 năm 2026
Workspace	D:\ABOUT_ROBOCON_2026\robocon-2026
Commit repository chính	0b20696
Commit giao diện	8a5b587
Trạng thái	Tài liệu phản ánh file đang có trên đĩa, bao gồm các thay đổi chưa commit trong repository chính

Kết luận chính Hệ thống đã tách rõ điều khiển chuyển động trên Teensy và điều khiển cơ cấu trên ESP32. Cả hai firmware cùng giao diện đều build thành công tại thời điểm lập báo cáo. Tuy nhiên source chính đang

có nhiều thay đổi chưa commit, một số state và hằng số kế thừa không còn nằm trong tuyến AUTO đang chạy, và tài liệu giao diện còn hai nhãn lưu EEPROM không đúng với hành vi RAM only của Teensy. Các điểm này cần được quản lý trước khi chốt bản thi đấu.

Mục lục

Mục lục.....	3
1 Phạm vi và trạng thái xác minh.....	5
1 1 Các giới hạn của kết quả xác minh.....	5
2 Kiến trúc hệ thống.....	5
3 Bố cục source code.....	6
4 Phần cứng và sơ đồ chân.....	7
4 1 Chân Teensy 4 1.....	7
4 2 Chân ESP32 S3.....	7
5 Firmware Teensy 4 1.....	8
5 1 Khởi tạo.....	8
5 2 Vòng lặp hợp tác.....	8
5 3 Động học Mecanum và giới hạn.....	8
5 4 Vòng PID tốc độ bốn bánh.....	8
5 5 Các vòng điều khiển ngoài.....	9
5 6 BNO085 và nhận diện dốc cầu.....	9
5 7 Cảm biến line.....	9
5 8 VL53L3CX.....	10
5 9 Odometry và MOVE.....	10
5 10 Nút vật lý và buzzer.....	10
5 11 Fault và watchdog.....	10
6 Luồng AUTO hiện tại.....	12
6 1 State kế thừa không nằm trong tuyến START hiện tại.....	13
6 2 Hằng số được định nghĩa nhưng không ảnh hưởng tuyến chính.....	14
7 Giao tiếp giữa Teensy và ESP32.....	14
8 Firmware ESP32 S3 cơ cấu.....	15
8 1 Cấu trúc chạy.....	15
8 2 Cấu hình profile.....	15
8 3 Tính vị trí và đồng bộ hai trục.....	15
8 4 Manual jog.....	15
8 5 Van và interlock.....	16
8 6 NVS và CRC.....	16
8 7 Fault cơ cấu.....	16
9 Firmware ESP32 giả lập.....	16
10 Giao diện Web Serial.....	16
10 1 Công nghệ và kết nối.....	16
10 2 Các trang chức năng.....	17

10 3 Cơ chế chống lag.....	17
10 4 Ghi log và xuất Excel.....	17
11 Danh mục lệnh điều khiển.....	17
12 Trình tự vận hành và kiểm tra.....	19
12 1 Trước khi cấp nguồn.....	19
12 2 Khởi động bằng giao diện.....	19
12 3 Khởi động không telemetry.....	19
12 4 Quy trình tune.....	19
13 Điểm cần kiểm soát trước khi chốt bản thi đấu.....	20
14 Ma trận tham số chính.....	20
15 Phụ lục danh mục hàm.....	21
Teensy main cpp.....	21
Teensy Esp32MechanismClient cpp.....	24
Teensy MechanismTelemetryView cpp.....	25
Teensy RBT2 Protocol cpp.....	25
ESP32 main cpp.....	25
ESP32 MechanismController cpp.....	25
ESP32 PersistentConfig cpp.....	26
ESP32 StepperProfiles cpp.....	26
ESP32 UartProtocol cpp.....	27
ESP32 ValveController cpp.....	27
ESP32 simulator main cpp.....	28
Giao diện page tsx.....	28
Giao diện manual jog.....	30
16 Phụ lục toàn bộ hằng số RobotConfig.....	31
17 Thư viện và môi trường build.....	39
18 Quy tắc kiểm soát thay đổi.....	40

1 Phạm vi và trạng thái xác minh

Báo cáo này mô tả source đang hoạt động trong workspace, không chỉ mô tả commit Git gần nhất. Cách đọc này cần thiết vì firmware cơ cấu ESP32 và nhiều mô đun mới của Teensy đang ở trạng thái chưa được commit trong repository chính. Các số liệu build bên dưới được kiểm tra trực tiếp từ source hiện tại.

Khối	Kết quả build	Thông tin chính
Teensy 4.1	Thành công	Platform Teensy 5.1.0; code 156176 byte; data 44540 byte; RAM1 còn 300864 byte; RAM2 còn 511872 byte
ESP32 S3 cơ cấu	Thành công	Espressif32 6.11.0; RAM 20596 trên 327680 byte; flash 328325 trên 3342336 byte
Giao diện	Thành công	vinext build hoàn tất cho route gốc

1 1 Các giới hạn của kết quả xác minh

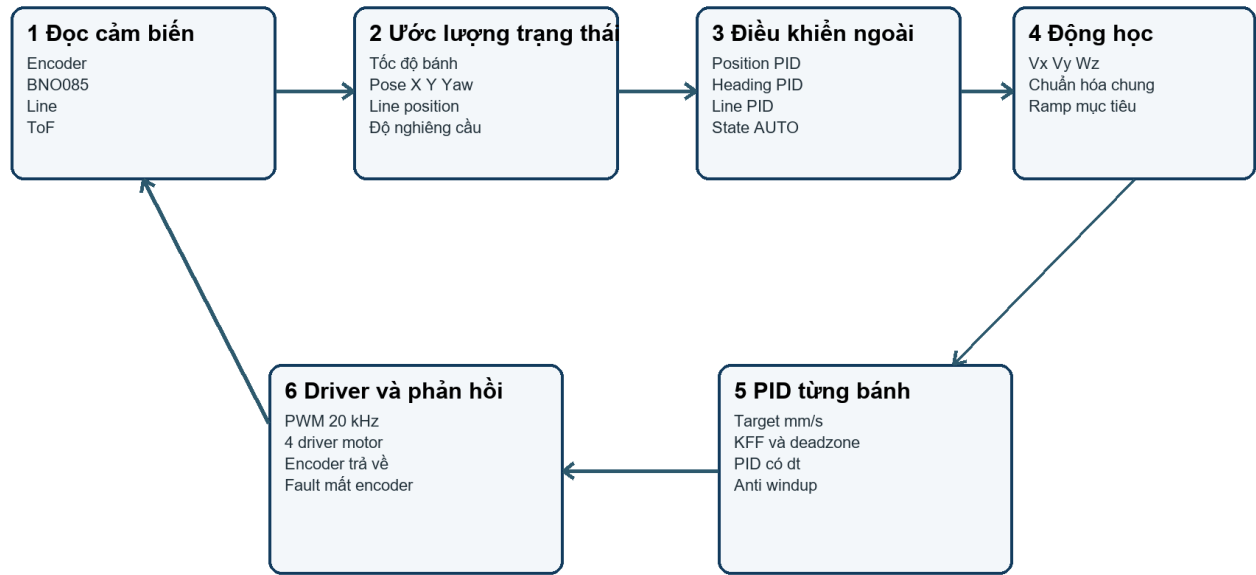
- Build thành công xác nhận cú pháp, liên kết thư viện và kích thước chương trình; build không thay thế thử nghiệm phần cứng dưới tải.
- Không có phép đo mới về chiều motor, cực tính cảm biến, khoảng cách thực tế hoặc đáp ứng PID trong quá trình lập tài liệu.
- Firmware có cơ chế bảo vệ bằng fault, watchdog và giới hạn phần mềm, nhưng E STOP vật lý độc lập chưa xuất hiện trong source Teensy hiện tại; biến dừng hiện tại là chốt E STOP do lệnh serial.
- Map_do là project ESP32 cơ cấu thật hiện tại. ESP32_MECHANISM_SIMULATOR là project thay thế để giả lập DONE sau 4 giây.

2 Kiến trúc hệ thống

Máy tính mở giao diện trong Chrome hoặc Edge và trao đổi với Teensy qua Web Serial. Teensy chịu trách nhiệm thời gian thực cho chuyển động, cảm biến, odometry và tuyến AUTO. ESP32 nhận lệnh cơ cấu từ Teensy qua UART riêng, phát xung cho hai động cơ bước, điều khiển năm van và trả trạng thái hoặc DONE.

Liên kết	Đầu thứ nhất	Đầu thứ hai	Tốc độ	Nội dung
Telemetry	Máy tính COM5	Teensy Serial5 RX21 TX20	57600 8N1	RBT 1 ASCII, lệnh điều khiển, cấu hình, telemetry
Cơ cấu	Teensy Serial6 TX24 RX25	ESP32 UART2 RX16 TX15	57600 8N1	Lệnh profile, jog, van, config, STATUS, DONE, ERR
Bảo trì Teensy	Máy tính COM14	USB native Teensy	115200	Đường nhận lệnh dự phòng; phản hồi chính vẫn được in qua Serial5 do macro Serial
Bảo trì ESP32	Máy tính USB	USB Serial ESP32	115200	Dùng cùng parser với UART2 và nhận bản sao phản hồi

Chu kỳ điều khiển Teensy



Các khối 1 đến 6 được cập nhật theo chu kỳ 10 ms; UART, ToF và buzzer chạy không chặn ngoài nhánh điều khiển.

Hình 2 Các tầng điều khiển trong chu kỳ 10 ms

3 Bố cục source code

Đường dẫn logic	Vai trò
firmwave ROBOCON KOSEN F0 include RobotConfig h	Cấu hình tập trung Teensy gồm chân, dấu chiều, PID, line, ToF, AUTO, UART và an toàn
firmwave ROBOCON KOSEN F0 src main cpp	Firmware thi đấu Teensy gồm toàn bộ vòng điều khiển và state machine
firmwave ROBOCON KOSEN F0 src mechanism	Client UART6 và bộ giải mã trạng thái cơ cấu ESP32
firmwave ROBOCON KOSEN F0 src communication	Khung nhị phân RBT 2 có CRC; hiện không được nối vào main RBT 1 ASCII
firmwave ROBOCON KOSEN F0 src READ và TUNE	Các firmware độc lập để đọc line, ToF, optical flow, encoder và tune bốn bánh
firmwave Map_do	Firmware ESP32 S3 cơ cấu thật gồm controller, profile, van, UART và NVS
firmwave ESP32_MECHANISM_SIMULATOR	Firmware giả lập nhận lệnh và trả DONE sau 4 giây
pid-tuner app page tsx	Giao diện chính, Web Serial, parser, log chuyển động, xuất Excel và các trang chức năng
pid-tuner app MechanismJog tsx	Hai cần nhấn giữ cho động cơ bước A và B
pid-tuner app mechanism-jog-controller ts	Bộ phát latest state only 120 ms và hủy lệnh cũ để giảm lag
pid-tuner worker và vite config	Lớp deploy Cloudflare và tích hợp Sites

4 Phần cứng và sơ đồ chân

4.1 Chân Teensy 4.1

Khối	Chân	Ghi chú
Motor FL	RPWM 36 LPWM 37	PWM 20 kHz
Motor FR	RPWM 22 LPWM 23	MOTOR SIGN bằng âm
Motor BL RL	RPWM 18 LPWM 19	Kênh sau đã được đổi nhận dạng vật lý
Motor BR RR	RPWM 14 LPWM 15	Kênh sau đã được đổi nhận dạng vật lý
Encoder FL	A0 B1	ENCODER SIGN âm
Encoder FR	A6 B7	ENCODER SIGN dương
Encoder BL RL	A2 B3	ENCODER SIGN dương
Encoder BR RR	A4 B5	ENCODER SIGN âm
MCP3008 center	CS26 SPI0 11 12 13	8 mắt giữa CH0 đến CH7
MCP3008 stop	CS8 SPI0 11 12 13	6 mắt dừng cùng H1 H2 CH6 CH7
I2C Wire1	SDA17 SCL16	BNO085 0x4A và VL53L3CX 0x29 dùng chung bus 100 kHz
Telemetry	RX21 TX20	Serial5 57600
ESP32	TX24 RX25	Serial6 57600
Nút START	34	Active low với INPUT PULLUP
Nút đổi map	35	Một nhấn RED hai nhấn BLUE
Buzzer	40	Active high qua AO3400A

4.2 Chân ESP32 S3

Khối	GPIO	Ghi chú
UART2	RX16 TX15	Nối chéo với Teensy TX24 RX25 và chung GND
Stepper A	STEP1 DIR2	AccelStepper DRIVER
Stepper B	STEP45 DIR48	GPIO45 là strap và GPIO48 dùng chung LED trên một số board
Van 1 đến 5	4 5 6 7 8	Active high, safe mask 0; source yêu cầu xác nhận dây trước khi cấp khí
Home và cảm biến cơ cấu	Chưa gán	HOME chỉ về zero phần mềm; E18 và line input đang bằng âm một

5 Firmware Teensy 4 1

5 1 Khởi tạo

1. Mở USB Serial 115200, cấu hình Serial5 telemetry và Serial6 cơ cấu.
2. Khởi tạo buzzer và hai nút active low.
3. Khởi tạo SPI, hai chân CS và bốn cặp PWM ở 20 kHz.
4. Nạp PID, KFF và deadzone đã biên dịch từ RobotConfig h vào RAM.
5. Sao chép min max hiệu chuẩn line vào mảng runtime.
6. Khởi tạo Wire1 100 kHz, thử BNO085 tối đa ba lần và đặt yaw tương đối về zero.
7. Khởi tạo VL53L3CX ở long distance, timing budget 100 ms và bắt đầu đo liên tục.
8. Reset pose, phát thông tin boot và giữ robot không chạy cho đến khi có lệnh hợp lệ.

5 2 Vòng lặp hợp tác

Hàm loop không dùng delay trong đường chạy bình thường. Mỗi vòng lặp cập nhật parser USB và telemetry, UART ESP32, optical flow, BNO085, ToF, nút và buzzer. Cứ đủ 10000 micro giây, firmware đọc line, encoder, odometry, kiểm tra fault, cập nhật test hoặc AUTO rồi chạy PID bánh. Telemetry được xử lý sau khối điều khiển.

loop -> UART -> ESP32 -> Flow -> BNO085 -> ToF -> Nút và buzzer -> chu kỳ 10 ms -> Telemetry

5 3 Động học Mecanum và giới hạn

Động học Mecanum cấu hình X

$FL = V_x - V_y - k W_z$	Kích thước và giới hạn $k = 0.5$ nhân chiều dài cộng chiều rộng k hiện tại bằng 305 mm Nếu bánh vượt giới hạn, cả 4 target cùng được chia một tỷ lệ Tỷ lệ hướng chuyển động được giữ nguyên
$FR = V_x + V_y + k W_z$	
$BL = V_x + V_y - k W_z$	
$BR = V_x - V_y + k W_z$	

Hình 3 Công thức target bốn bánh

Trong công thức, V_x dương là tiến, V_y dương là sang phải theo quy ước nội bộ và W_z dương là quay theo chiều kim đồng hồ. k bằng nửa tổng khoảng cách tâm bánh trước sau và trái phải, hiện bằng 305 mm. Nếu một target vượt giới hạn bánh, setRobotVelocity chia đồng thời cả bốn target cho cùng một hệ số. Sau đó target được ramp đồng bộ với gia tốc 1400 mm trên giây bình phương và giảm tốc 2400 mm trên giây bình phương, nhờ đó tỷ lệ hướng Mecanum không bị phá bởi việc kẹp từng bánh độc lập.

5 4 Vòng PID tốc độ bốn bánh

Bánh	Kp	Ki	Kd	KFF	Deadzone PWM	Motor sign	Encoder sign
FL	0.15	0.08	0	0.035	36	+1	-1

Bánh	Kp	Ki	Kd	KFF	Deadzone PWM	Motor sign	Encoder sign
FR	0.14	0.08	0	0.045	34	-1	+1
BL RL	0.14	0.12	0	0.034	34	+1	+1
BR RR	0.15	0.10	0	0.034	34	-1	-1

Encoder được đổi từ count sang milimet bằng chu vi bánh 100 mm chia 300 count mỗi vòng. Tốc độ đo được lọc IIR với alpha 0.35. PWM bằng KFF nhân target cộng deadzone theo dấu, sau đó cộng hiệu chỉnh PID. PID dùng dt, giới hạn tích phân 250 và anti windup theo bão hòa đầu ra. Nếu target lớn hơn 100 mm trên giây nhưng encoder không đổi trong 600 ms, firmware chốt FAULT ENCODER và dừng.

5 5 Các vòng điều khiển ngoài

Vòng	Hệ số hiện tại	Vai trò
Heading thường	Kp 0.101 Ki 0 Kd 0.004	Giữ yaw; deadband 0.50 độ; Wz tối thiểu 0.18; max 1.2 rad trên giây
Heading fine	Kp 0.120 Kd 0.003	Căn góc chính xác; deadband 0.25 độ; max 0.35
Heading ngang	Fine PID cộng slew	Deadband 0.45 độ; min 0.06; max 0.20; slew 0.80 rad trên giây bình phương
Heading tại trạm	Fine PID cộng slew	Deadband 0.45 độ; min 0.05; max 0.16; slew 0.55
Position X và Y	Mỗi trục 1.40 0.05 0.01	MOVE tương đối và GOTO tuyệt đối; ngưỡng hoàn thành 18 mm
Line center	0.16 0.02 0.003	Sai số weighted của 8 mắt thành Vy; giảm Vx khi line lệch về mép
ToF	0.80 0 0	Căn khoảng cách target 160 mm; hiện không chặn A và B
Stop forward	0.20 0.01 0.002	Căn theo trung bình vị trí hai cụm trong hàm stop alignment kế thừa
Stop yaw	0.0018 0.0001 0.00002	Căn chênh lệch hai cụm line kế thừa
Cụm 3 mắt trái phải	Kp 0.08	Kéo cặp mắt mục tiêu về vị trí 500 tại B

5 6 BNO085 và nhận diện dốc cầu

BNO085 phát Game Rotation Vector mỗi 10 ms. Firmware đổi quaternion thành yaw, roll và pitch. Yaw được đặt tương đối theo yawReferenceDeg; tốc độ quay được lọc. Roll và pitch dùng alpha 0.35. Trên cầu, robot chụp roll và pitch nền lúc bắt đầu bám line, chọn trục nghiêng lớn hơn khi độ nghiêng vượt 5 độ trong 150 ms, tăng tốc lên 1300 mm trên giây khi đang leo, nhận đỉnh khi trục nghiêng trở lại dưới 4 độ trong 120 ms sau tối thiểu 300 ms leo, rồi giảm còn 400 mm trên giây để xuống.

5 7 Cảm biến line

Cụm	Kênh vật lý	Thứ tự logic	Công dụng
8 mắt giữa	MCP CS26 CH0 đến CH7	Trọng số âm 3500 đến dương 3500	Bám line lên cầu và nhận vạch ngang khi ít nhất 6 mắt active
3 mắt trái	MCP CS8 CH3 CH4 CH5	Ngoài âm 1000 giữa 0 trong dương 1000	Căn điểm theo map

Cụm	Kênh vật lý	Thứ tự logic	Công dụng
3 mắt phải	MCP CS8 CH0 CH1 CH2	Ngoài âm 1000 giữa 0 trong dương 1000	RED dừng tại A; cụm đối diện dừng tại B
H1 H2	MCP CS8 CH6 CH7	H1 ở đuôi H2 ở đầu	Giữ trôi dọc khi chạy ngang và tìm line dọc

Line đen cho ADC cao hơn nền trắng. Dữ liệu được chuẩn hóa 0 đến 1000 và cảm biến được xem là HIGH trên line khi normalized đạt ngưỡng. Ngưỡng chung là 550; riêng phát hiện nhanh pick line dừng 400. Center có min max đã hiệu chuẩn; sáu mắt stop cũng có min max đã hiệu chuẩn. H1 H2 vẫn dùng min 0 max 1023 mặc định và nên được hiệu chuẩn riêng để tăng độ tin cậy.

5 8 VL53L3CX

VL53L3CX dùng địa chỉ 0x29 trên Wire1 chung với BNO085. Firmware chọn vật thể hợp lệ gần nhất trong vùng 35 đến 800 mm, loại bước nhảy lớn hơn 250 mm và lọc median 5 mẫu. Timeout mẫu là 1500 ms. Tại A và B, ToF chỉ được phát bản tin thông tin và không quyết định chuyển state. Target vẫn là 160 mm, dung sai 30 mm. Luồng xuất phát hiện tại tắt nhận ToF trong đoạn tiến 170 mm vì cảm biến ở quá gần tường.

5 9 Odometry và MOVE

Odometry Mecanum tính dx và dy trong thân robot từ bốn delta encoder, dùng BNO085 làm yaw khi hợp lệ và xoay delta sang hệ tọa độ thế giới. Optical flow đang tắt, do đó trọng số odometry hiện là encoder 1.0 và flow 0.0. MOVE nhận dx dy tương đối trong hệ tọa độ tại thời điểm bắt đầu. GOTO nhận X Y yaw tuyệt đối trong hệ pose hiện tại. Cả hai đều yêu cầu BNO085 và heartbeat; mất heartbeat sau 600 ms sẽ dừng.

5 10 Nút vật lý và buzzer

Tác động	Xử lý
Nhấn START chân 34	Kêu 1 tiếng, kiểm tra idle, E STOP, fault, BNO và cấu hình; tự chọn AUTO rồi vào CAN START dù không có telemetry
Nhấn map chân 35 một lần	Sau cửa sổ 450 ms chọn sân RED và kêu 2 tiếng
Nhấn map hai lần	Chọn sân BLUE và kêu 3 tiếng
Đến A hoặc B và bắt đầu chờ cơ cấu	Kêu 2 tiếng
Fault trong nhóm state cần A hoặc B	Buzzer chớp tắt 120 ms trong 2 giây

Hai nút dùng INPUT PULLUP nên thả là HIGH và nhấn nối GND là LOW. Debounce bằng thời gian, buzzer dùng hàng đợi và không chặn vòng điều khiển.

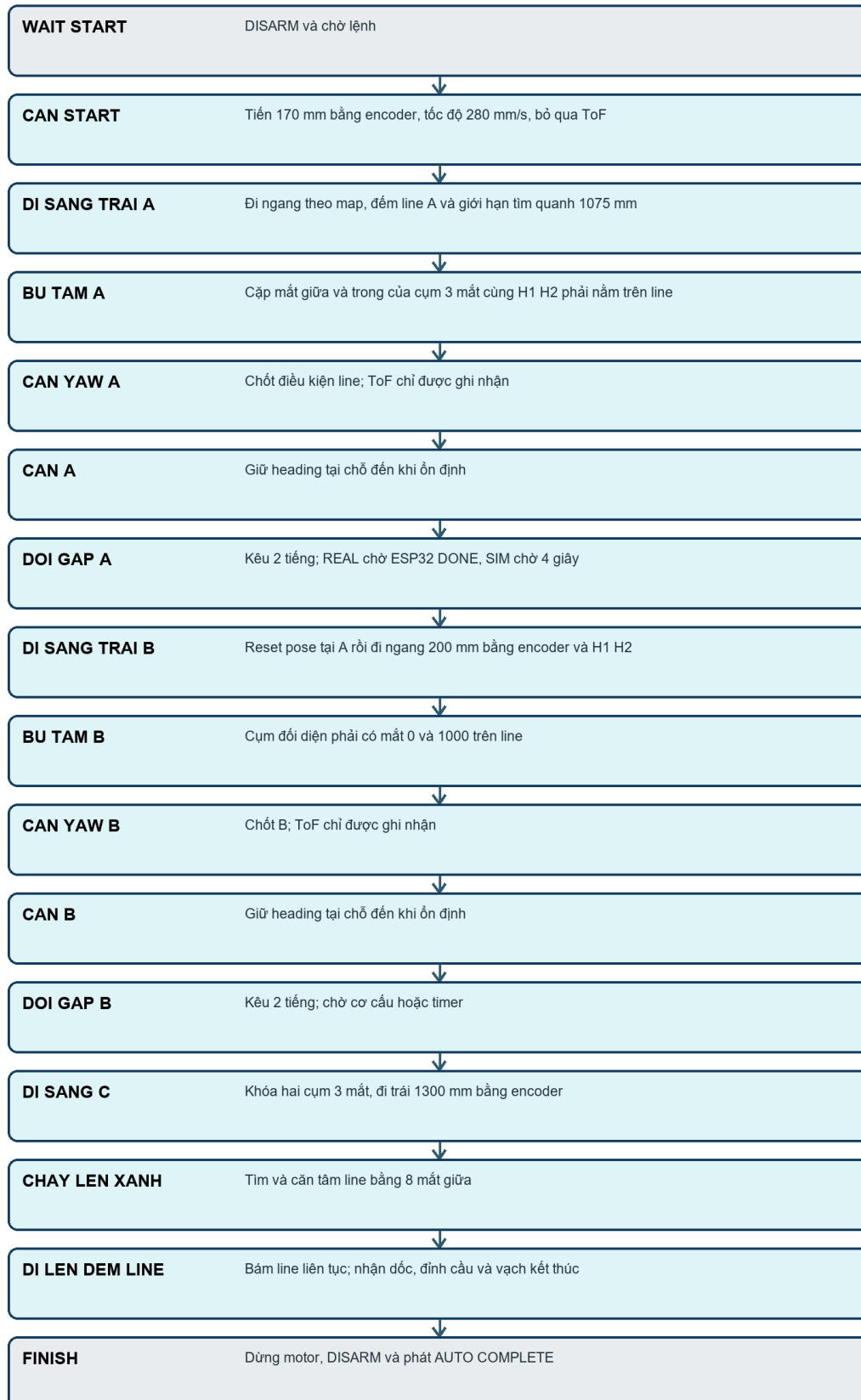
5 11 Fault và watchdog

Bit	Tên	Nguyên nhân điển hình	Tác động
0	IMU TIMEOUT	BNO không có mẫu quá 250 ms	AUTO dừng; manual DRIVE có thể bỏ qua riêng bit này
1	FLOW TIMEOUT	Optical flow mất dữ liệu	Hiện flow tắt nên không dùng trong tuyến chính
2	LINE LOST	Mất center line quá 650 ms hoặc H1 H2 mất quá thời gian	Dừng và FAULT STOP

Bit	Tên	Nguyên nhân điển hình	Tác động
3	TOF TIMEOUT	ToF bắt buộc ở hàm căn ToF nhưng mất quá 3 s	Dừng
4	ENCODER	Có target trên 100 nhưng encoder đứng 600 ms	Dừng bốn bánh
5	ESP TIMEOUT	Cơ cấu REAL không trả lời trong state chờ	Dừng AUTO
6	STATE TIMEOUT	State vượt thời gian cho phép	Dừng AUTO
7	CONFIG	Cấu hình không đủ hoặc tune bị khóa	Không cho motor chạy

6 Luồng AUTO hiện tại

Luồng AUTO đang được gọi từ START



Hình 4 Tuyến AUTO thực sự được gọi từ START

Sân RED đặt dấu chạy ngang là dương và dùng cụm 3 mắt phải tại A. Sân BLUE đảo dấu chạy ngang và dùng cụm trái tại A. B dùng cụm đối diện. Heading cạnh tranh được chụp đúng lúc START và giữ xuyên suốt A và B.

State	Hành động	Điều kiện hoàn tất	State sau
WAIT START	Robot DISARM	START UART hoặc nút 34	CAN START
CAN START	Tiến 170 mm encoder ở 280	MOVE hoàn thành	DI SANG TRAI A
DI SANG TRAI A	Tìm pick line A quanh 1075 cộng trừ 200 mm	Đếm đủ 2 line, encoder trong cửa sổ, mắt giữa và trong đã đi qua	BU TAM A
BU TAM A	Kéo chậm về line	Mắt giữa và trong của cụm A cùng H1 H2 ổn định 180 ms	CAN YAW A
CAN YAW A	Xác nhận line và ghi ToF	Line còn đúng	CAN A
CAN A	Căn heading tại chỗ	X Y đã chốt và heading ổn định	DOI GAP A
DOI GAP A	Gửi POINT A hoặc SIM POINT A	DONE hoặc đủ 4 s	DI SANG TRAI B
DI SANG TRAI B	Reset pose rồi chạy ngang 200 mm	Relative move hoàn thành	BU TAM B
BU TAM B	Căn cụm đối diện	Mắt 0 và dương 1000 cùng nằm trên line 180 ms	CAN YAW B
CAN YAW B	Chốt B và ghi ToF	Ngay sau ghi	CAN B
CAN B	Căn heading tại chỗ	Heading ổn định	DOI GAP B
DOI GAP B	Gửi POINT B hoặc SIM POINT B	DONE hoặc đủ 4 s	DI SANG C
DI SANG C	Khóa cụm 3 mắt và đi trái 1300 mm ở 800	MOVE hoàn thành	CHAY LEN XANH
CHAY LEN XANH	Quét trái phải trong bán kính 180 mm và căn center line	Line hợp lệ và gần tâm 180 ms	DI LEN DEM LINE
DI LEN DEM LINE	Bám line, tăng giảm ga theo dốc	Sau đỉnh cầu gấp vạch ngang ít nhất 80 ms	FINISH
FINISH	PWM về zero, DISARM	Lệnh RESET hoặc START mới	WAIT START qua lệnh

6 1 State kế thừa không nằm trong tuyến START hiện tại

Các state sau vẫn còn trong enum và switch nhưng không được tuyến START hiện tại chuyển tới. Chúng là phần kế thừa hoặc đường thử. Không nên chỉnh tốc độ trong các state này với kỳ vọng tuyến thi đấu sẽ thay đổi:

- CAN START FAST
- CAN START FINE
- BU TAM C
- CAN YAW C
- CAN GIUA LINE C
- HOME CENTER
- HOME DROP
- HOME TO B
- TIEN 34CM B
- DOI THA 2B
- CHO SAU B

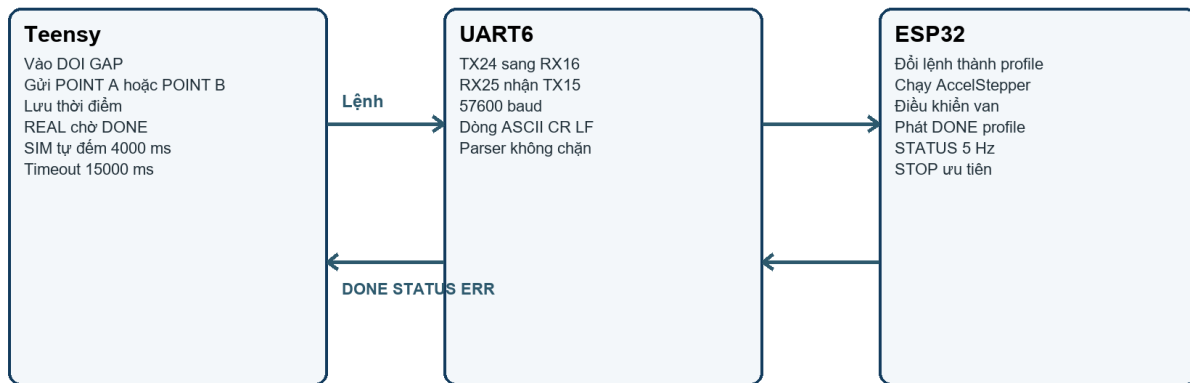
- LUI 33CM A
- DOI THA 2A
- CHO SAU THA A
- LUI 3M
- SANG TRAI 1M5

6 2 Hằng số được định nghĩa nhưng không ảnh hưởng tuyến chính

Hằng số	Giá trị	Nhận xét
START LATERAL DISTANCE MM	1000	Không có tham chiếu trong main hiện tại
START LATERAL SPEED MM S	400	Không có tham chiếu trong main hiện tại
TOF START ESCAPE DISTANCE MM	250	Không có tham chiếu trong main hiện tại
TOF START ESCAPE SPEED MM S	300	Không có tham chiếu trong main hiện tại
PICK B LINE TARGET	1	Tuyến B hiện dùng encoder 200 mm rồi căn cặp mắt, không đếm line
BRIDGE LINE TARGET	3	Không được dùng; tuyến hiện dùng theo center cross sau đỉnh
BRIDGE SPEED FAST SLOW	1150 và 1000	Chỉ còn tham chiếu trong file recovery, không được main dùng
BRIDGE ACCEL DECEL	1800 và 700	Chỉ còn tham chiếu trong file recovery

7 Giao tiếp giữa Teensy và ESP32

Bắt tay cơ cấu tại điểm A và B



Trong SIM 4S, Teensy gửi SIM lệnh để ESP32 không tác động cơ cấu. Teensy tự chuyển bước khi đủ 4 giây.

Hình 5 Cơ chế chờ cơ cấu tại A và B

Chế độ REAL là mặc định sau mỗi lần reset Teensy. Ở chế độ này, robot chỉ rời state chờ khi nhận chuỗi chứa DONE. Nếu sau 5.5 giây chưa DONE, Teensy gửi lại POINT A hoặc POINT B với khoảng cách giữa hai lần gửi ít nhất 5 giây. Timeout state là 15 giây. Chế độ SIM 4S gửi khung SIM để ESP32 xác nhận nhưng không chạy cơ cấu; Teensy tự coi hoàn thành khi đủ 4000 ms.

Teensy gửi	ESP32 cơ cấu thật	Phản hồi quyết định
POINT A	Chạy profile PICK A	DONE PICK A
POINT B	Chạy profile PICK B	DONE PICK B
THA 2A hoặc THA 2B	Chạy profile RETRACT	DONE RETRACT
SIM lệnh	Chỉ ACK NO MOTION	Teensy tự hết 4 giây
STOP	Dừng stepper và đưa van về safe mask	ACK STOPPED SAFE

8 Firmware ESP32 S3 cơ cấu

8 1 Cấu trúc chạy

main.cpp chỉ khởi tạo USB Serial, UART2, đọc cấu hình NVS hoặc nạp mặc định, khởi tạo MechanismController và gọi protocol update liên tục. UartProtocol đọc đồng thời UART2 và USB bằng hai buffer cố định 256 byte. Mỗi lần update, controller được gọi trước và sau khi đọc lệnh để AccelStepper run có cơ hội phát xung đều.

8 2 Cấu hình profile

Profile	A mm	B mm	Dir A	Dir B	Speed step/s	Accel step/s ²	Chờ ms	Mode
ROBOT START	10	0	-1	+1	5000 5000	3500 3500	300	Tương đối
PICK A	500	500	-1	-1	5000 5000	3500 3500	3000	Tương đối
PICK B	615	500	-1	+1	5000 5000	3500 3500	3000	Tương đối
RETRACT	2200	2200	+1	-1	5000 5000	3500 3500	3000	Tương đối
HOME	0	0	+1	+1	5000 5000	3500 3500	0	Tuyệt đối

Mỗi profile mặc định có ba bước: phát lệnh move, chờ hai stepper về target, rồi chờ post wait. Khung dữ liệu cho phép tối đa tám bước và enum đã hỗ trợ bật van, chờ thời gian, chờ stepper, chờ cảm biến và kết thúc. Tuy nhiên profile mặc định hiện không có bước ValveSet hoặc WaitSensor.

8 3 Tính vị trí và đồng bộ hai trục

Số step trên milimet bằng step trên vòng nhân hệ số hiệu chỉnh rồi chia bước vít. Mặc định A dùng 200 step trên vòng và vít 12 mm trên vòng; B dùng 200 và 10. Khi hai trục có quãng đường khác nhau, firmware chỉ giảm tốc trục ngắn hơn để thời gian chạy gần bằng trục dài hơn, không tăng quá giới hạn đã cấu hình. Chuyển profile vẫn giữ tọa độ milimet bằng cách đổi currentPosition theo scale mới.

8 4 Manual jog

JOG nhận tốc độ có dấu riêng cho A và B trong vùng 10 đến 30000 step trên giây. Giao diện gửi lại mỗi 120 ms. ESP32 yêu cầu có lệnh mới trong 500 ms; quá thời gian sẽ dừng và chốt JogLinkTimeout. Đổi chiều làm

tốc độ dư được xóa trước khi ramp lại. Lệnh JOG STOP dừng hai trục nhưng không phát DONE và không thay đổi van.

8 5 Van và interlock

Năm van active high khởi động ở safe mask 0. writeOne ghi mức safe trước khi đổi chân sang OUTPUT để giảm xung kích lúc boot. Interlock hiện đều bằng 0, nghĩa là source chưa cấm cặp van nào bật cùng lúc. Nếu mạch dùng hai coil đối nghịch, cần điền VALVE INTERLOCK MASK trước khi chạy khí thật.

8 6 NVS và CRC

MechanismConfig có magic MECH, version, size, giới hạn vị trí, năm profile và CRC32. SAVE kiểm tra phạm vi, tính CRC, ghi namespace mechanism key config, đọc lại toàn bộ, so sánh byte và kiểm CRC lần nữa. Nếu load thất bại, ESP32 nạp mặc định vào RAM nhưng không tự ghi NVS. RESTORE DEFAULT cũng chỉ nạp RAM cho đến khi có SAVE.

8 7 Fault cơ cấu

Mã enum	Tên	Ý nghĩa
0	None	Không lỗi
1	Busy	Dùng cho dừng bất thường có reason khác STOP hoặc BOOT SAFE
2	InvalidProfile	Profile không hợp lệ
3	PositionLimit	Target vượt giới hạn phần mềm
4	MotionTimeout	Profile hoặc direct move quá deadline
5	SensorUnavailable	Profile yêu cầu sensor chưa gán chân
6	SensorTimeout	Sensor không đạt trạng thái đúng hạn
7	ValveInterlock	Hai van xung đột
8	NotZeroed	HOME khi chưa SET ZERO và không có công tắc home
9	JogLinkTimeout	Mất refresh JOG trên 500 ms

9 Firmware ESP32 giả lập

Project ESP32 MECHANISM SIMULATOR dùng cùng UART2 RX16 TX15 ở 57600. Nó nhận POINT A, POINT B, THA 2A, THA 2B hoặc SIM lệnh. Với lệnh hợp lệ, timer không chặn bắt đầu, LED bật và sau 4 giây gửi đúng một dòng DONE. Lệnh lặp khi bạn bị bỏ qua nên không làm timer chạy lại. STOP hoặc ESTOP hủy pending ngay. Firmware này không điều khiển stepper hoặc van.

10 Giao diện Web Serial

10 1 Công nghệ và kết nối

Giao diện dùng React 19, Next 16, TypeScript, vinext và Vite. Bản deploy chạy qua Cloudflare Worker và Sites. Web Serial mở cổng ở baud do người dùng chọn, mặc định 57600 và buffer 8192 byte. Khi kết nối, giao diện gửi PING, GET CONFIG, MECH GET STATUS và MECH GET CONFIG. Nếu chỉ nhận mà không có chiều gửi, giao diện cảnh báo kiểm tra TXD bộ chuyển sang RX5 chân 21.

10 2 Các trang chức năng

Trang	Chức năng chính
Tổng quan	Hiển thị bốn bánh, heading, line và ToF
Điều khiển Mecanum	DRIVE nhấn giữ, heading manual, MOVE tương đối, pose, log và tải Excel
AUTO Setup sân	Chọn map, REAL hoặc SIM 4S, telemetry mode, GOTO, snapshot, START, DISARM, E STOP
CƠ CẤU ESP32	Profile, trạng thái, hai cần jog, năm van, SET ZERO, config RAM và SAVE NVS
PID 4 bánh	Chọn một bánh, PID, target RPM, chạy thuận nghịch và heartbeat an toàn
Heading PID	PID heading, giới hạn Wz, deadband, Wz tối thiểu và giới hạn tốc độ bánh
Line PID	Đọc raw normalized, hiệu chuẩn 8 mắt, 6 mắt và xem H1 H2
Cấu hình	GET CONFIG, SAVE CONFIG, PROFILE LOAD và FACTORY RESET
Chẩn đoán	Trạng thái liên kết, RX TX, state, fault và ARM

10 3 Cơ chế chống lag

Parser đọc mọi dòng để ghi log, nhưng UI chỉ render theo batch 40 ms. Các record tần suất cao như WHEEL được ghi đè bằng bản mới nhất theo key; record sự kiện như STATE, ACK và ERR vẫn giữ thứ tự. Các dòng không cần cho giao diện như MOTION, DBG, TILT, STOP6, HOLD2, PICK SCAN và BRIDGE SCAN bị bỏ khỏi hàng render. Hàng lệnh serial được tuần tự hóa. DRIVE và JOG dùng latest state only để không tích tụ lệnh cũ phía sau thao tác ghi chậm.

10 4 Ghi log và xuất Excel

Giao diện tự ghi mẫu khi MOTION báo active. Mỗi mẫu giữ thời gian PC và robot, mode, map, state, fault, ARM, Vx Vy Wz, sai số vị trí, pose, heading, ToF và target actual error PWM encoder của từng bánh. File tải về là workbook XML Excel có sheet Tổng quan và Dữ liệu 4 bánh. Tốc độ thực tế robot được suy ra từ RPM bốn bánh và đường kính 100 mm; đây là tốc độ suy ra từ encoder, không phải tốc độ đo độc lập ngoài robot.

11 Danh mục lệnh điều khiển

Lệnh Teensy	Chức năng
PING	Bắt tay và nhận HELLO
GET CONFIG	Đọc PID, giới hạn, mode, map và cấu hình AUTO
HEARTBEAT	Giữ tune, MOVE, GOTO hoặc test giám sát
MODE MANUAL hoặc AUTO	Đổi mode khi DISARM
FIELD RED hoặc BLUE	Chọn map khi DISARM
START hoặc RUN	Chạy tuyến AUTO hiện tại
START TEST	AUTO có heartbeat giám sát
START BRIDGE TEST	Bắt đầu từ đoạn sau B để thử cầu
ARM	Chỉ test khóa heading AUTO, không chạy tuyến

Lệnh Teensy	Chức năng
STOP hoặc DISARM	Dừng và về WAIT START
ESTOP	Chốt E STOP phần mềm và gửi STOP sang ESP32
RESET	Xóa E STOP phần mềm và fault
DRIVE vx vy wz	Điều khiển vận tốc manual; phải refresh trong 500 ms
MOVE dx dy maxSpeed	Vị trí tương đối
GOTO x y yaw maxSpeed	Vị trí tuyệt đối ở mode AUTO
POSE RESET	Đặt pose về zero khi DISARM
MANUAL HEADING ON OFF	Bật tắt giữ heading manual
MANUAL YAW degrees	Đặt yaw manual
PID WHEEL name kp ki kd	Đổi PID bánh trong RAM
SET PID index kp ki kd kff deadzone	Đổi đầy đủ điều khiển một bánh trong RAM
PID HEADING	Đổi PID heading trong RAM
PID LINE	Đổi PID line trong RAM
LIMITS HEADING	Đổi max Wz, deadband và min Wz khi DISARM
LIMITS MOTOR	Đổi giới hạn bánh 100 đến 2000 mm trên giây
TUNE WHEEL	Quay một bánh theo target RPM
TUNE STOP	Dừng tune
TEST LINE CENTER STOP	Bật dữ liệu raw line tương ứng
CAL CENTER hoặc CAL STOP	Bắt đầu và kết thúc hiệu chuẩn
AUTO MECH REAL hoặc SIM 4S	Chọn cơ cấu thật hoặc giả lập
AUTO TELEM ON REQUEST hoặc SILENT	Chọn chính sách telemetry AUTO
AUTO TELEM SNAPSHOT	Lấy một snapshot phân mảnh
MECH command	Chuyển command nguyên văn xuống ESP32

Lệnh ESP32	Chức năng
PING	HELLO ESP32 MECHANISM
GET STATUS	Trạng thái, vị trí, van, profile, step, zero và jog
GET CONFIG	Header CRC và năm profile
CMD profile	Chạy ROBOT START, PICK A, PICK B, RETRACT hoặc HOME
POINT A POINT B	Alias trực tiếp cho PICK A và PICK B
THA 2A THA 2B	Alias trực tiếp cho RETRACT
JOG speedA speedB	Điều khiển giữ tốc độ hai trục
JOG STOP	Dừng manual jog

Lệnh ESP32	Chức năng
VALVE n state	Điều khiển van 1 đến 5 khi idle
STOP ESTOP CMD STOP	Dừng ưu tiên và safe off van
SET ZERO	Đặt tọa độ phần mềm bằng zero
HOME	Về zero phần mềm
SET PROFILE	Cập nhật một profile trong RAM
SAVE	Ghi NVS và kiểm CRC
RESTORE DEFAULT	Nạp mặc định vào RAM
CLEAR FAULT	Xóa fault khi không bận
NANG HA SPEED POS	Tương thích lệnh USB cũ
SIM action	Xác nhận không chạy cơ cấu

12 Trình tự vận hành và kiểm tra

12 1 Trước khi cấp nguồn

- Kiểm tra GND chung, TX RX nối chéo, dây BNO, ToF, hai MCP3008 và encoder.
- Xác nhận vùng chạy trống, nguồn motor và nguồn van có nút ngắt độc lập, E STOP sẵn sàng.
- Đặt cơ cấu trong vùng hành trình an toàn vì chưa có công tắc home hoặc limit trong source.
- Đặt robot ở đúng vạch xuất phát và kiểm tra H1 H2 cùng các cụm 3 mắt theo map.

12 2 Khởi động bằng giao diện

- Mở giao diện bằng Chrome hoặc Edge, chọn KẾT NỐI TEENSY và COM5 ở 57600 baud.
- Chờ HELLO, GET CONFIG và MECH STATUS. Nếu cơ cấu offline, kiểm tra UART6.
- Chọn AUTO, map RED hoặc BLUE, chọn REAL hoặc SIM 4S và chính sách telemetry.
- Nếu dùng cơ cấu thật, đọc config ESP32, xác nhận zero và profile trước khi chạy.
- Giữ robot DISARM và nhấn START AUTO ĐẾN CẦU. Không nhấn ARM trước vì ARM chỉ test khóa đầu.
- Theo dõi sự kiện STATE, FAULT và DONE. Dùng snapshot khi cần nhưng không stream AUTO.

12 3 Khởi động không telemetry

- Chờ BNO085 khởi tạo và đảm bảo robot không có fault chót từ lần chạy trước.
- Nhấn nút map chân 35 một lần cho RED hoặc hai lần nhanh cho BLUE; nghe 2 hoặc 3 tiếng xác nhận.
- Nhấn START chân 34 một lần; nghe một tiếng. Firmware tự chuyển AUTO và chạy nếu đủ điều kiện.
- Nếu còi cảnh báo chớp trong 2 giây, dừng cấp lực và kết nối telemetry để đọc fault.

12 4 Quy trình tune

- Tune bốn bánh khi kê khỏi sàn trước, sau đó kiểm tra lại có tải ở tốc độ thấp, trung bình và cao.
- Xác nhận chiều motor và encoder trước khi thay Kp Ki Kd KFF hoặc deadzone.
- Tune KFF và deadzone để đạt gần target, sau đó tăng Kp, thêm Ki vừa đủ và chỉ dùng Kd khi dữ liệu tốc độ đủ sạch.
- Tune heading sau khi bốn bánh đồng đều. Tune line sau khi heading ổn và min max cảm biến đúng.
- Mọi PID Teensy chỉnh từ giao diện chỉ nằm trong RAM. Chép giá trị đã chót vào RobotConfig h rồi build lại.

13 Điểm cần kiểm soát trước khi chốt bản thi đấu

Mức	Vấn đề	Tác động	Hướng xử lý
Cao	Repository chính đang dirty và Map_do chưa được theo dõi	Có thể mất hoặc nhầm phiên bản source	Commit theo mốc có tag và ghi checksum firmware hex
Cao	Không có E STOP vật lý độc lập trong source Teensy	Mất serial hoặc treo MCU có thể không nhận lệnh ESTOP	Bổ sung ngõ dừng phần cứng hoặc mạch cắt enable nguồn motor
Cao	ESP32 chưa có limit home switch	HOME dựa trên zero phần mềm và mất mốc sau sự cố	Gắn limit, homing từng trục và kiểm tra polarity
Cao	GPIO van và interlock chưa xác minh	Có thể kích sai van hoặc hai coil đối nghịch	Đo từng ngõ ra khi chưa cấp khí rồi điền interlock
Trung bình	H1 H2 dùng min max mặc định 0 1023	Ngưỡng thực tế có thể ít biên chống nhiễu	Hiệu chuẩn riêng H1 H2 và lưu hằng số
Trung bình	Nhiều state và hằng số kế thừa không được dùng	Người chỉnh nhầm tham số nhưng robot không đổi	Xóa hoặc gắn nhãn legacy rõ trong code
Trung bình	RBT 2 có source nhưng main dùng RBT 1 ASCII	Hai giao thức dễ gây hiểu nhầm	Quyết định giữ RBT 1 hoặc hoàn tất tích hợp RBT 2
Trung bình	Nhãn EEPROM trên UI và protocol sai	Người vận hành tưởng PID Teensy đã lưu bền	Đổi nhãn thành RAM only và sửa tài liệu protocol
Trung bình	USB COM14 nhận lệnh nhưng phản hồi dùng macro Serial5	Khó tune chỉ bằng USB	Tách hàm emit theo cổng nguồn lệnh hoặc ghi rõ giới hạn
Thấp	README Teensy còn chi tiết VL53L1X và tuyến cũ	Tài liệu cục bộ không khớp source	Cập nhật README từ báo cáo này

14 Ma trận tham số chính

Nhóm	Tham số	Giá trị	Ảnh hưởng
Kích thước	WHEEL DIAMETER MM	100	Quy đổi encoder và RPM
Kích thước	ROBOT LENGTH MM	260	k Mecanum
Kích thước	ROBOT WIDTH MM	350	k Mecanum
Encoder	COUNTS PER WHEEL REV	300	mm per count
Motor	MAX WHEEL SPEED	1300 mm/s	Trần target bánh
Motor	TARGET ACCEL DECEL	1400 2400	Khởi động và dừng mềm
Chu kỳ	CONTROL PERIOD	10000 us	100 Hz
Heading	KP KI KD	0.101 0 0.004	Giữ yaw
Heading	LATERAL MAX WZ	0.20 rad/s	Đi ngang A B
Line	ACTIVE NORMALIZED	550	Logic HIGH trên line đen
Line	PICK DETECT	400	Phát hiện line hẹp tốc độ cao
Line	CENTER CONTROL SIGN	-1	Chiều hiệu chỉnh 8 mắt
H1 H2	CONTROL SIGN	+1	Chiều kéo đuôi và đầu về line

Nhóm	Tham số	Giá trị	Ảnh hưởng
ToF	TARGET	160 mm	Thông tin tại A B; dừng trong state cần ToF nếu gọi
ToF	VALID RANGE	35 đến 800 mm	Lọc target
Xuất phát	START ENCODER	170 mm ở 280 mm/s	Bỏ qua ToF
A	EXPECTED LATERAL	1075 cộng trừ 200 mm	Cửa sổ tìm line
A đến B	DISTANCE	200 mm ở 300 mm/s	Relative move
Sau B	POST B LEFT	1300 mm ở 800 mm/s	Tìm line cầu
Cầu	APPROACH ASCENT DESCENT	1000 1300 400 mm/s	Theo phase độ nghiêng
UART	TELEMETRY PERIOD	200 ms	5 Hz ngoài AUTO ARM
UART	MANUAL WATCHDOG	500 ms	Dừng DRIVE
UART	POSITION WATCHDOG	600 ms	Dừng MOVE GOTO
Cơ cấu	SIM DELAY	4000 ms	AUTO mô phỏng
Cơ cấu	REPLY TIMEOUT	15000 ms	FAULT ESP

15 Phụ lục danh mục hàm

Các bảng sau được trích tự động từ source hiện tại. Số dòng giúp tra cứu trong VS Code; số dòng có thể thay đổi sau khi sửa code.

Teensy main cpp

Dòng	Hàm	Tham số
20	beginUsbCommandStream	
236	selectedFieldLateralSign	
241	selectedFieldUsesRightStopGroup	
247	secondPickUsesRightStopGroup	
436	applyFactoryPidConfig	
456	wrap180	float angle
462	shortestAngleError	float target, float current
466	readFloatLE	const uint8_t *p
472	readU32LE	const uint8_t *p
482	stationAlignmentInProgress	
489	setFault	FaultCode fault, const char *reason
505	stopInputActive	
509	clearFaults	

Dòng	Hàm	Tham số
517	hardwareConfigurationValid	
522	competitionConfigurationValid	
531	wheelTuneConfigurationValid	
536	configurationAllowsMotion	
545	writeMotor	WheelControl &wheel, int pwm
551	resetWheelPid	WheelControl &wheel
559	resetAllControllers	
577	stopRobot	
586	setRobotVelocity	float vx, float vy, float wz
631	updateEncoderMeasurements	float dt
644	updateWheelSpeedPid	float dt
672	enableBnoReport	
680	initializeBno085	
694	quaternionYaw	float real, float i, float j, float k
699	quaternionRoll	float real, float i, float j, float k
704	quaternionPitch	float real, float i, float j, float k
709	updateBno085	
781	headingControl	float dt, float limit = MAX_HEADING_WZ_RAD_S
799	headingControlFine	float dt, float limit = FINE_HEADING_MAX_WZ_RAD_S
813	headingControlLateral	float dt
841	headingControlStationary	float dt
873	readMCP3008Raw	uint8_t csPin, uint8_t channel
882	readMCP3008	uint8_t csPin, uint8_t channel
889	normalizeLine	uint16_t raw, uint16_t minimum, uint16_t maximum
897	updateLineArrays	
989	updateLineFollowing	float dt, float &vx
1014	printLineCalibration	
1026	printCenterCalibrationTelemetry	
1035	printStopCalibrationTelemetry	
1122	updateOpticalFlow	
1145	medianTof	

Dòng	Hàm	Tham số
1156	updateTof	
1197	resetPose	float x=0, float y=0, float yaw=0
1206	updateOdometry	float dt
1231	lateralXHoldVelocity	
1257	resetLateralLineHold	
1264	lateralLineHoldVelocity	float dt
1301	firstLateralGuidanceReady	
1324	beginRelativeMove	float dx, float dy, float maxSpeed=MOVE_SPEED_MM_S
1336	beginAbsoluteMove	float targetX,float targetY,float targetYaw,float maxSpeed
1345	updateAbsoluteMove	float dt
1380	updateRelativeMove	float dt
1403	updateRelativeLateralMoveWithHold	float dt
1424	updateRelativeMoveWithLine	float dt
1442	updateBridgeLineUntilEndMarker	float dt
1512	updateYawAlignment	float dt, float tolerance=1.0f
1520	updateTofAlignment	float dt
1561	beginStopAlignment	float searchVy,uint8_t crossings,bool rightGroup
1569	updateStopAlignment	float dt
1688	stopGroupActiveCount	uint8_t offset
1695	pointARequiredSensorsOnLine	bool rightGroup
1708	updatePointAPassMemory	bool rightGroup
1716	updateAutoPickTravel	float dt,bool rightGroup,uint8_t targetCount
1800	updateAutoPickX	float dt,bool rightGroup
1896	autoHeadingAligned	
1906	autoPickVerified	
1912	reportStationTofInfo	const char *point
1950	sendEsp	const char *command
1970	printAutoStartConfiguration	const char *source
1976	printAutoTelemetryMode	
1981	requestAutoTelemetrySnapshot	
1994	printAutoMechanismStatus	
2006	printMechanismTelemetryStatus	

Dòng	Hàm	Tham số
2019	beginAutoMechanismAction	const char *command
2037	autoMechanismActionComplete	
2058	handleMechanismLine	const char *line, void *
2073	updateEspProtocol	
2098	tunerWheelIndex	const char *name
2106	wheelRpmToMmS	float rpm
2107	wheelMmSToRpm	float speed
2109	sendTunerConfig	
2138	enterPassiveTest	TestMode mode
2142	processCommand	const char *line
2516	updateUsbCommands	
2525	updateTestMode	float dt
2630	onExitState	AutoState
2632	transitionTo	AutoState next
2638	onEnterState	
2784	checkStateTimeout	
2794	updateCompetition	float dt
3023	printAutoTelemetrySnapshotStage	
3065	printTelemetry	
3211	confirmPhysicalField	FieldSide field
3218	startAutoFromPhysicalButton	
3258	updatePhysicalControls	
3291	setup	
3370	loop	

Teensy Esp32MechanismClient cpp

Dòng	Hàm	Tham số
3	Esp32MechanismClient::begin	uint8_t rxPin, uint8_t txPin, uint32_t baud
11	Esp32MechanismClient::send	const char *line
15	Esp32MechanismClient::update	LineHandler handler, void *context

Teensy MechanismTelemetryView cpp

Dòng	Hàm	Tham số
6	MechanismTelemetryView::consume	const char *line
33	MechanismTelemetryView::updateTimeout	uint32_t timeoutMs

Teensy RBT2 Protocol cpp

Dòng	Hàm	Tham số
12	packetCrc	const uint8_t *packet, size_t length
27	crc16Ccitt	const uint8_t *data, size_t length, uint16_t initial
37	encodePacket	PacketType type, uint32_t sequence, uint32_t timestampMs, const void *payload, uint16_t payloadLength, uint8_t *output,
64	StreamParser::StreamParser	PacketHandler handler, void *context) : handler_(handler), context_(context
67	StreamParser::reset	
73	StreamParser::beginHeader	
81	StreamParser::feed	uint8_t byte
108	StreamParser::feed	const uint8_t *data, size_t length
113	StreamParser::validateHeader	
136	StreamParser::finishPacket	

ESP32 main cpp

Dòng	Hàm	Tham số
23	setup	
42	loop	

ESP32 MechanismController cpp

Dòng	Hàm	Tham số
9	MechanismController::MechanismController	AccelStepper &motorA, AccelStepper &motorB, ValveController &valves) : motorA_(motorA), motorB_(motorB), valves_(valves
14	MechanismController::begin	MechanismConfig &config
27	MechanismController::stepsPerMmA	const StepperProfile &p
31	MechanismController::stepsPerMmB	const StepperProfile &p
35	MechanismController::positionAmm	
41	MechanismController::positionBmm	

Dòng	Hàm	Tham số
47	MechanismController::selectProfileScale	ProfileId id
59	MechanismController::targetsWithinLimits	float targetAmm, float targetBmm
68	MechanismController::issueProfileMotion	const StepperProfile &p
103	MechanismController::startProfile	ProfileId id
117	MechanismController::startDirectMove	float signedAmm, float signedBmm, float speedStepsPerSecond
142	MechanismController::applyJogAxis	AccelStepper &motor, uint8_t axis, float speed, long target, float acceleration
160	MechanismController::setJog	float signedSpeedA, float signedSpeedB
193	MechanismController::stopJog	
205	MechanismController::advanceStep	
210	MechanismController::sensorValue	uint8_t sensorIndex, bool &available
221	MechanismController::update	
290	MechanismController::setValve	uint8_t index, bool enabled
299	MechanismController::setSoftwareZero	
306	MechanismController::startHome	
314	MechanismController::emergencyStop	const char *reason
328	MechanismController::clearFault	
336	MechanismController::completeOperation	
342	MechanismController::setFault	MechanismFault fault, const char *reason
356	MechanismController::takeDoneEvent	char *profileText, size_t capacity
366	MechanismController::takeFaultEvent	char *reason, size_t capacity

ESP32 PersistentConfig.cpp

Dòng	Hàm	Tham số
9	setReason	char *reason, size_t capacity, const char *text
16	PersistentConfig::begin	
21	PersistentConfig::load	MechanismConfig &config, char *reason, size_t reasonCapacity
47	PersistentConfig::save	MechanismConfig &config, char *reason, size_t reasonCapacity
76	PersistentConfig::erase	char *reason, size_t reasonCapacity

ESP32 StepperProfiles.cpp

Dòng	Hàm	Tham số
10	initProfile	StepperProfile &profile, const char *name, float distanceA, float distanceB, int8_t directionA, int8_t directionB, uint3
40	finiteInRange	float value, float minimum, float maximum
44	setReason	char *reason, size_t capacity, const char *text
58	parseProfileId	const char *text, ProfileId &id
69	loadDefaultMechanismConfig	MechanismConfig &config
90	mechanismConfigCrc	const MechanismConfig &config
102	validateMechanismConfig	const MechanismConfig &config, char *reason, size_t reasonCapacity

ESP32 UartProtocol.cpp

Dòng	Hàm	Tham số
20	uppercaseAscii	char *text
27	isSimulatedAutoAction	const char *action
33	UartProtocol::UartProtocol	Stream &usb, Stream &robot, MechanismController &controller, MechanismConfig &config, PersistentConfig &persistentConfig
40	UartProtocol::begin	
48	UartProtocol::emit	const char *line
53	UartProtocol::emitAck	const char *token
59	UartProtocol::emitError	const char *code, const char *detail
66	UartProtocol::readStream	Stream &stream, Receiver &receiver
89	UartProtocol::runNamedProfile	const char *name
102	UartProtocol::updateProfile	const char *arguments
146	UartProtocol::emitCapabilities	
150	UartProtocol::processJog	const char *arguments
173	UartProtocol::processLine	char *rawLine
242	UartProtocol::emitProfile	ProfileId id
255	UartProtocol::emitConfig	
267	UartProtocol::emitStatus	
278	UartProtocol::update	

ESP32 ValveController.cpp

Dòng	Hàm	Tham số
5	ValveController::writeOne	uint8_t index, bool enabled
11	ValveController::begin	
21	ValveController::set	uint8_t index, bool enabled
31	ValveController::setMask	uint8_t mask
43	ValveController::safeOff	

ESP32 simulator main.cpp

Dòng	Hàm	Tham số
24	isSupportedCommand	const char *command
48	startSimulatedAction	const char *command
89	processCompleteLine	
98	readRobotCommands	
120	finishSimulatedActionWhenDue	
142	setup	
158	loop	

Giao diện page tsx

Dòng	Hàm hoặc component	Tham số
96	escapeXml	value: unknown
100	excelCell	value: string number, style = ""
105	Pill	{ tone = "good", children }: { tone?: "good" "warn" "bad" "off"; children: React.ReactNode }
109	Gauge	{ current, target }: { current: number; target: number }
119	GainsEditor	{ gains, onChange }: { gains: Gains; onChange: (key: keyof Gains, value: number) => void }
123	SectionTitle	{ eyebrow, title, action }: { eyebrow: string; title: string; action?: React.ReactNode }
127	Home	
233	if	!manualDriveActiveRef.current && !heartbeatPendingRef.current && queuedWritesRef.current === 0
285	if	kind === "WHEEL"
321	if	["HELLO", "CFG", "CFG_END", "ACK", "ERR"].includes(fields[0])

Dòng	Hàm hoặc component	Tham số
325	if	fields[0] === "MECH_STATUS" && fields.length >= 13
329	if	[fault, positionA, positionB, valveMask, step].every(Number.isFinite)
338	if	fields[1] === "STATUS" && fields.length >= 11
341	if	[fault, positionA, positionB, valveMask, step].every(Number.isFinite)
358	if	mechanismProfileIds.includes(id)
360	if	values.every(Number.isFinite)
387	if	fields[1] === "MODE" && (fields[2] === "ON_REQUEST" fields[2] === "SILENT")
401	if	[fault, vx, vy, wz, x, y, poseYaw, actualYaw, wantedYaw, distance].every(Number.isFinite)
424	if	fields[1] === "FAULT"
457	if	values.every(Number.isFinite)
465	if	values.every(Number.isFinite)
471	if	values.every(Number.isFinite)
481	if	values.every(Number.isFinite)
487	if	values.every(Number.isFinite)
546	if	fields[1] === "RESET"
547	if	fields[1] === "ESTOP"
558	if	fields[0] === "WHEEL" && fields.length >= 6
563	if	id && [targetRpm, actualRpm, pwm, encoder].every(Number.isFinite)
581	if	values.every(Number.isFinite)
585	if	active
586	if	!motionLogActiveRef.current
595	if	now - motionLogCountUpdatedAtRef.current >= 500
614	if	!motionLogRef.current.length
669	while	portRef.current === port
671	if	value
676	for	const raw of lines
690	if	connected
709	if	!serial
741	if	!connected
748	if	!configSynced
755	if	!configSynced
756	if	robotArmed

Dòng	Hàm hoặc component	Tham số
758	if	manualWz < .05 manualWz > 1.5 autoWz < .05 autoWz > 2 deadbandDeg < 0 deadbandDeg > 5 minWz < 0 min
770	if	!configSynced
777	if	direction !== 0 && !wheelTuneReady
783	if	manualDriveTimerRef.current !== null
796	while	latestDriveRef.current !== null
808	if	!connected !configSynced
847	if	!mechanismConfigSynced
865	if	next === "mechanism"
877	if	autoTelemetryMode === "SILENT"
927	Overview	{ wheels, yaw, targetYaw, lineError, lineRawReady, tofDistance, tofValid, open }; { wheels: Wheel[]; yaw: number; target
935	MotionControl	{ connected, manualReady, positionReady, fault, pose, yaw, targetYaw, manualHeadingLock, setManualHeading, setManualYaw,
980	AutoControl	{ connected, ready, mode, field, state, armed, fault, estop, bnoValid, tofDistance, tofValid, tofTarget, yaw, targetYaw,
1036	MechanismNumber	{ label, value, unit, min, max, step = 1, disabled, onChange }: { label: string; value: number; unit: string; min?: numb
1112	Drive	{ wheels, selectedWheel, setSelectedWheel, changeGain, tuneTargetRpm, changeTarget, ready, sendPid, applyAll, command }:
1117	Heading	{ yaw, target, setTarget, gains, setGain, limits, setLimit, maxWheelSpeed, setMaxWheelSpeed, sendSettings, saveConfig, s
1127	Line	{ error, gains, centerRaw, centerNormalized, centerCalMin, centerCalMax, calibrationActive, centerValid, centerPosition,
1175	Config	{ send }: { send: (command: string) => void }
1180	Diagnostics	{ connected, linkAlive, rx, tx, state, fault, armed }: { connected: boolean; linkAlive: boolean; rx: number; tx: number;

Giao diện manual jog

Dòng	Hàm hoặc phương thức	Tham số
12	MechanismJog	{ ready, supported, connected, busy, positions, confirmedSpeeds, write }: Props
24	if	!ready
51	if	!Number.isFinite(speed[axis]) speed[axis] < 10 speed[axis] > 30000
81	if	event.key === " " event.key === "Enter"
10	constructor	private send: Send, private onError: () => void
12	setEnabled	enabled: boolean
17	setAxis	axis: 0 1, speed: number
23	if	!this.speeds.some(Boolean)

Dòng	Hàm hoặc phương thức	Tham số
27	stop	force = false
36	pump	

16 Phụ lục toàn bộ hằng số RobotConfig

Bảng này liệt kê mọi constexpr tìm thấy trong RobotConfig h để kiểm soát thay đổi. Giá trị mảng được giữ theo thứ tự source. Hãy ưu tiên bảng Ma trận tham số chính khi vận hành và dùng phụ lục này khi review code.

Dòng	Tên	Giá trị trong source
10	HARDWARE_CONFIG_VERIFIED	true
13	DEBUG_TELEMETRY	false
16	WHEEL_DIAMETER_MM	100.0f
17	ROBOT_LENGTH_MM	260.0f
18	ROBOT_WIDTH_MM	350.0f
19	ENCODER_COUNTS_PER_WHEEL_REV	300.0f
20	DISTANCE_CALIBRATION	1.0f
24	RED_FIELD_LATERAL_SIGN	1
25	BLUE_FIELD_LATERAL_SIGN	-1
27	MOTOR_SIGN_FL	1
28	MOTOR_SIGN_FR	-1
31	MOTOR_SIGN_RL	1
32	MOTOR_SIGN_RR	-1
34	ENCODER_SIGN_FL	-1
36	ENCODER_SIGN_FR	1
37	ENCODER_SIGN_RL	1
38	ENCODER_SIGN_RR	-1
42	MECANUM_X_CONFIGURATION	true
45	FL_RPWM	36, FL_LPWM = 37
46	FR_RPWM	22, FR_LPWM = 23
47	RL_RPWM	18, RL_LPWM = 19
48	RR_RPWM	14, RR_LPWM = 15
50	FL_ENC_A	0, FL_ENC_B = 1
51	FR_ENC_A	6, FR_ENC_B = 7
52	RL_ENC_A	2, RL_ENC_B = 3

Dòng	Tên	Giá trị trong source
53	RR_ENC_A	4, RR_ENC_B = 5
55	PWM_FREQUENCY_HZ	20000
56	PWM_MAX	255
57	MAX_WHEEL_SPEED_MM_S	1300.0f
58	WHEEL_SPEED_FILTER_ALPHA	0.35f
59	CONTROL_PERIOD_US	10000
62	WHEEL_TARGET_ACCEL_MM_S2	1400.0f
63	WHEEL_TARGET_DECEL_MM_S2	2400.0f
70	WHEEL_KP[4]	{ 0.15f, 0.14f, 0.14f, 0.15f }
76	WHEEL_KI[4]	{ 0.08f, 0.08f, 0.12f, 0.10f }
82	WHEEL_KD[4]	{ 0.0f, 0.0f, 0.0f, 0.0f }
88	WHEEL_KFF[4]	{0.035f, 0.045f, 0.034f, 0.034f}
89	WHEEL_DEADZONE_PWM[4]	{36, 34, 34, 34}
90	WHEEL_INTEGRAL_LIMIT	250.0f
93	POSITION_X_KP	1.40f
94	POSITION_X_KI	0.05f
95	POSITION_X_KD	0.01f
96	POSITION_X_INTEGRAL_LIMIT	400.0f
97	POSITION_Y_KP	1.40f
98	POSITION_Y_KI	0.05f
99	POSITION_Y_KD	0.01f
100	POSITION_Y_INTEGRAL_LIMIT	400.0f
101	STOP_FORWARD_KP	0.20f
102	STOP_FORWARD_KI	0.01f
103	STOP_FORWARD_KD	0.002f
104	STOP_FORWARD_INTEGRAL_LIMIT	600.0f
105	STOP_YAW_KP	0.0018f
106	STOP_YAW_KI	0.0001f
107	STOP_YAW_KD	0.00002f
108	STOP_YAW_INTEGRAL_LIMIT	500.0f
109	TOF_DISTANCE_KP	0.80f
110	TOF_DISTANCE_KI	0.0f
111	TOF_DISTANCE_KD	0.0f
112	TOF_DISTANCE_INTEGRAL_LIMIT	300.0f

Dòng	Tên	Giá trị trong source
113	RIGHT_LINE_KP	0.08f
114	RIGHT_LINE_KI	0.0f
115	RIGHT_LINE_KD	0.0f
116	RIGHT_LINE_INTEGRAL_LIMIT	1000.0f
117	LEFT_LINE_KP	0.08f
118	LEFT_LINE_KI	0.0f
119	LEFT_LINE_KD	0.0f
120	LEFT_LINE_INTEGRAL_LIMIT	1000.0f
124	BNO085_I2C_ADDRESS	0x4A
125	SENSOR_I2C_HZ	100000
126	BNO_USE_GAME_ROTATION_VECTOR	true
130	BNO_YAW_SIGN	1
131	BNO_REPORT_INTERVAL_US	10000
132	BNO_TIMEOUT_MS	250
133	BNO_RETRY_INTERVAL_MS	1000
135	HEADING_KP	0.101f
136	HEADING_KI	0.0f
137	HEADING_KD	0.004f
138	HEADING_INTEGRAL_LIMIT	30.0f
139	MAX_HEADING_WZ_RAD_S	1.2f
140	MAX_MANUAL_HEADING_WZ_RAD_S	0.60f
141	MAX_AUTO_HEAD_LOCK_WZ_RAD_S	1.00f
142	MAX_LINE_ALIGN_WZ_RAD_S	0.45f
143	MAX_BRIDGE_HEADING_WZ_RAD_S	0.25f
144	HEADING_OUTPUT_DEADBAND_DEG	0.50f
145	HEADING_MIN_WZ_RAD_S	0.18f
146	FINE_HEADING_KP	0.120f
147	FINE_HEADING_KD	0.0030f
148	FINE_HEADING_DEADBAND_DEG	0.25f
149	FINE_HEADING_MIN_WZ_RAD_S	0.180f
150	FINE_HEADING_MAX_WZ_RAD_S	0.35f
154	STATION_HEADING_DEADBAND_DEG	0.45f
155	STATION_HEADING_MIN_WZ_RAD_S	0.05f
156	STATION_HEADING_MAX_WZ_RAD_S	0.16f

Dòng	Tên	Giá trị trong source
157	STATION_HEADING_SLEW_RAD_S2	0.55f
158	STATION_HEADING_SETTLED_RATE_DEG_S	2.0f
161	LATERAL_HEADING_DEADBAND_DEG	0.45f
162	LATERAL_HEADING_MIN_WZ_RAD_S	0.06f
163	LATERAL_HEADING_MAX_WZ_RAD_S	0.20f
164	LATERAL_HEADING_SLEW_RAD_S2	0.80f
165	LATERAL_LINE_COUNT_MAX_YAW_ERROR_DEG	2.0f
166	LATERAL_HEADING_RECOVERY_SPEED_FACTOR	0.45f
169	MCP_CENTER_CS	26
170	MCP_STOP_CS	8
173	STOP_LEFT_CHANNELS[3]	{3, 4, 5}
174	STOP_RIGHT_CHANNELS[3]	{0, 1, 2}
175	MCP_SPI_HZ	1000000
179	LINE_IS_HIGHER_THAN_FLOOR	true
180	CENTER_SENSOR_MIN[8]	{84,79,76,75,74,75,81,83}
181	CENTER_SENSOR_MAX[8]	{1023,1023,1023,1023,969,866,1023,1023}
183	STOP_SENSOR_MIN[6]	{76,76,76,80,83,81}
184	STOP_SENSOR_MAX[6]	{1023,1023,1023,1023,1023,1023}
187	LATERAL_HOLD_SENSOR_MIN[2]	{0, 0}
188	LATERAL_HOLD_SENSOR_MAX[2]	{1023, 1023}
190	LATERAL_HOLD_TAIL_CHANNEL	6
191	LATERAL_HOLD_FRONT_CHANNEL	7
192	LATERAL_HOLD_LINE_THRESHOLD	500
195	LATERAL_HOLD_CONTROL_SIGN	1
196	LATERAL_HOLD_CORRECTION_SPEED_MM_S	65.0f
197	LATERAL_HOLD_SEARCH_SPEED_MM_S	35.0f
198	LATERAL_HOLD_SLEW_MM_S2	220.0f
199	LATERAL_HOLD_LOST_TIMEOUT_MS	450
200	LATERAL_HOLD_ACQUIRE_TIMEOUT_MS	1200
204	FIRST_LATERAL_BLIND_DISTANCE_MM	400.0f
205	CENTER_LINE_CALIBRATION_VERIFIED	true
206	STOP_LINE_CALIBRATION_VERIFIED	true
209	STOP_SENSOR_ORDER_VERIFIED	true

Dòng	Tên	Giá trị trong source
210	LINE_FILTER_ALPHA	0.55f
211	LINE_ACTIVE_NORMALIZED	550
212	CROSS_MIN_ACTIVE_SENSORS	6
213	CROSS_CONFIRM_MS	35
214	CROSS_RELEASE_MS	100
215	LINE_SHORT_LOST_MS	180
216	LINE_FAULT_TIMEOUT_MS	650
217	LINE_KP	0.16f
218	LINE_KI	0.02f
219	LINE_KD	0.003f
221	CENTER_LINE_CONTROL_SIGN	-1
224	MAX_LINE_VY_MM_S	220.0f
225	LINE_LOST_SEARCH_VY_MM_S	140.0f
228	STOP_WEIGHTS[3]	{-1000, 0, 1000}
229	STOP_POSITION_TOLERANCE	120.0f
230	STOP_SIDE_DIFFERENCE_TOLERANCE	150.0f
231	STOP_ALIGNMENT_CONFIRM_MS	180
232	STOP_CROSS_CONFIRM_MS	40
233	STOP_CROSS_RELEASE_MS	100
241	PICK_A_LINE_TARGET	2
242	PICK_B_LINE_TARGET	1
245	BRIDGE_LINE_TARGET	3
246	LINE_DEBOUNCE_MS	40
247	LINE_CLEAR_DEBOUNCE_MS	100
250	PICK_LINE_DETECT_NORMALIZED	400
251	PICK_LINE_CONFIRM_MS	8
252	PICK_LINE_CLEAR_MS	60
253	X_ALIGN_STABLE_TIME_MS	180
254	LINE_SEARCH_TIMEOUT_MS	5000
255	HEADING_STABLE_TIME_MS	350
256	HEADING_TOLERANCE_DEG	0.60f
257	MOVE_LEFT_SPEED_MM_S	400.0f
261	LATERAL_LONGITUDINAL_HOLD_ENABLED	false
263	LATERAL_X_HOLD_KP	1.20f

Dòng	Tên	Giá trị trong source
264	LATERAL_X_HOLD_DEADBAND_MM	6.0f
265	LATERAL_X_HOLD_MAX_SPEED_MM_S	140.0f
268	LATERAL_TOF_HOLD_KP	0.90f
269	LATERAL_TOF_HOLD_DEADBAND_MM	8.0f
270	LATERAL_TOF_HOLD_MAX_SPEED_MM_S	130.0f
272	BRIDGE_LATERAL_SCAN_SPEED_MM_S	500.0f
273	X_ALIGN_MAX_SPEED_MM_S	80.0f
274	X_ALIGN_SEARCH_SPEED_MM_S	35.0f
277	POINT_B_PAIR_TARGET_POSITION	500.0f
278	POINT_B_PAIR_SEARCH_TIMEOUT_MS	10000
279	TOF_ALIGN_MAX_SPEED_MM_S	100.0f
280	BRIDGE_FORWARD_SPEED_MM_S	1300.0f
281	BRIDGE_APPROACH_SPEED_MM_S	1000.0f
282	BRIDGE_DESCENT_SPEED_MM_S	400.0f
285	BRIDGE_INCLINE_ENTER_DEG	5.0f
286	BRIDGE_CREST_LEVEL_DEG	4.0f
287	BRIDGE_INCLINE_CONFIRM_MS	150
288	BRIDGE_CREST_CONFIRM_MS	120
289	BRIDGE_MIN_ASCENT_BEFORE_CREST_MS	300
290	BRIDGE_END_MARKER_CONFIRM_MS	80
291	BNO_TILT_FILTER_ALPHA	0.35f
292	BRIDGE_LINE_SEARCH_RADIUS_MM	180.0f
293	BRIDGE_LINE_SEARCH_SPEED_MM_S	60.0f
294	BRIDGE_ENTRY_CENTER_KP	0.08f
296	BRIDGE_ENTRY_CENTER_MIN_VY_MM_S	90.0f
297	BRIDGE_ENTRY_CENTER_MAX_VY_MM_S	140.0f
298	BRIDGE_ENTRY_CENTER_TOLERANCE	300.0f
299	BRIDGE_ENTRY_CENTER_CONFIRM_MS	80
302	BRIDGE_STOP_CROSS_TARGET	3
303	BRIDGE_LINE_ACQUIRE_CONFIRM_MS	180
304	BRIDGE_LINE_ACQUIRE_TIMEOUT_MS	10000
305	BRIDGE_FOLLOW_TIMEOUT_MS	20000
307	TOF_CONTROL_SIGN	1
310	LEFT_STOP_CONTROL_SIGN	1

Dòng	Tên	Giá trị trong source
311	RIGHT_STOP_CONTROL_SIGN	-1
317	OPTICAL_FLOW_ENABLED	false
319	FLOW_BAUD	115200
320	MAVLINK_OPTICAL_FLOW_RAD_ID	106
321	MAVLINK_OPTICAL_FLOW_RAD_LEN	44
322	MAVLINK_OPTICAL_FLOW_RAD_CRC_EXTRA	138
323	FLOW_X_FROM_SENSOR_AXIS	0
324	FLOW_Y_FROM_SENSOR_AXIS	1
325	FLOW_X_SIGN	1
326	FLOW_Y_SIGN	1
327	FLOW_SCALE_X_MM	1.0f
328	FLOW_SCALE_Y_MM	1.0f
329	FLOW_MIN_QUALITY	80
330	FLOW_TIMEOUT_MS	250
331	ODOM_ENCODER_WEIGHT	1.0f
332	ODOM_FLOW_WEIGHT	0.0f
333	LATERAL_SLIP_THRESHOLD_MM_S	180.0f
338	TOF_ENABLED	true
339	TOF_I2C_ADDRESS	0x29
341	TOF_TARGET_MM	160
342	TOF_MIN_VALID_MM	35
343	TOF_MAX_VALID_MM	800
345	TOF_MAX_SAMPLE_JUMP_MM	250
346	TOF_TIMEOUT_MS	1500
349	TOF_ALIGNMENT_RECOVERY_MS	3000
353	START_ENCODER_DISTANCE_MM	170.0f
354	START_ENCODER_SPEED_MM_S	280.0f
355	START_LATERAL_DISTANCE_MM	1000.0f
356	START_LATERAL_SPEED_MM_S	400.0f
357	START_HOLD_LINE_SEARCH_SPEED_MM_S	280.0f
358	START_HOLD_LINE_CONFIRM_MS	100
359	START_HOLD_LINE_SEARCH_TIMEOUT_MS	8000
362	POINT_A_RIGHT_ALIGN_SPEED_MM_S	50.0f
363	POINT_A_RIGHT_SENSOR_THRESHOLD	500

Dòng	Tên	Giá trị trong source
364	POINT_A_RIGHT_CONFIRM_MS	120
365	POINT_A_RIGHT_PAIR_WINDOW_MS	1200
366	POINT_A_RIGHT_SEARCH_TIMEOUT_MS	15000
370	POINT_A_EXPECTED_LATERAL_MM	1075.0f
371	POINT_A_SEARCH_HALF_RANGE_MM	200.0f
372	POINT_A_SEARCH_SPEED_MM_S	70.0f
375	POINT_A_TO_B_DISTANCE_MM	200.0f
376	POINT_A_TO_B_SPEED_MM_S	300.0f
379	POINT_A_H_SEARCH_RADIUS_MM	100.0f
380	POINT_A_H_SEARCH_SPEED_MM_S	35.0f
381	POINT_A_H_SEARCH_TIMEOUT_MS	15000
383	POST_B_LEFT_DISTANCE_MM	1300.0f
384	POST_B_LEFT_SPEED_MM_S	800.0f
385	TOF_START_ESCAPE_DISTANCE_MM	250.0f
386	TOF_START_ESCAPE_SPEED_MM_S	300.0f
387	TOF_MEDIAN_SAMPLES	5
388	TOF_COARSE_SPEED_MM_S	180.0f
389	TOF_FINE_SPEED_MM_S	65.0f
390	TOF_TOLERANCE_MM	30.0f
391	TOF_CONFIRM_MS	200
397	TELEMETRY_BAUD	57600
398	TELEMETRY_RX_PIN	21
399	TELEMETRY_TX_PIN	20
403	ESP32_UART_ENABLED	true
405	ESP32_TX_PIN	24
406	ESP32_RX_PIN	25
407	ESP32_BAUD	57600
411	AUTO_MECHANISM_SIM_DELAY_MS	4000
414	ESP32_REPLY_TIMEOUT_MS	15000
415	STATE_DEFAULT_TIMEOUT_MS	12000
418	TELEMETRY_PERIOD_MS	200
419	TUNER_WATCHDOG_MS	600
420	MANUAL_DRIVE_WATCHDOG_MS	500
421	POSITION_LINK_WATCHDOG_MS	600

Dòng	Tên	Giá trị trong source
424	MAX_POSITION_HEADING_WZ_RAD_S	0.35f
428	PHYSICAL_CONTROL_BUTTONS_ENABLED	true
429	PHYSICAL_START_BUTTON_PIN	34
430	PHYSICAL_FIELD_BUTTON_PIN	35
431	PHYSICAL_BUTTON_DEBOUNCE_MS	35
432	PHYSICAL_FIELD_DOUBLE_CLICK_MS	450
435	BUZZER_PIN	40
436	BUZZER_ACTIVE_HIGH	true
437	BUZZER_BEEP_ON_MS	110
438	BUZZER_BEEP_OFF_MS	100
439	BUZZER_PATTERN_GAP_MS	180
440	BUZZER_ALARM_TOGGLE_MS	120
441	BUZZER_ALIGNMENT_ALARM_MS	2000
449	SENSOR_TO_CENTER_MM	20.0f
450	START_APPROACH_MM	125.0f
451	C_OFFSET_MM	150.0f
452	HOME_TO_B_MM	410.0f
453	RETURN_A_MM	330.0f
454	RETURN_LONG_MM	2650.0f
455	RETURN_LATERAL_MM	1650.0f
456	MOVE_SPEED_MM_S	450.0f
459	BRIDGE_SPEED_FAST_MM_S	1150.0f
460	BRIDGE_SPEED_SLOW_MM_S	1000.0f
461	BRIDGE_ACCEL_MM_S2	1800.0f
462	BRIDGE_DECEL_MM_S2	700.0f

17 Thư viện và môi trường build

Khối	Thư viện hoặc framework	Phiên bản hoặc vai trò
Teensy	Arduino Teensy	Framework 1.60 trên Teensy platform 5.1.0
Teensy	Encoder	1.4.4; đọc encoder quadrature
Teensy	Adafruit BNO08x	1.2.7; SH2 Game Rotation Vector
Teensy	PWFFusion VL53L3C	1.0.0; multi target ToF
Teensy	SPI và Wire	Thư viện lõi Arduino

Khối	Thư viện hoặc framework	Phiên bản hoặc vai trò
ESP32	Arduino ESP32	Platform Espressif32 6.11.0
ESP32	AccelStepper	1.64; phát xung không chặn
ESP32	Preferences	NVS thuộc Arduino ESP32
UI	React	19.2.6
UI	Next	16.2.6
UI	TypeScript	5.9.3
UI	vinext và Vite	vinext 0.0.50; Vite 8.0.13
UI	Cloudflare plugin	Deploy Worker và Sites

18 Quy tắc kiểm soát thay đổi

28. Mỗi lần chốt hoạt động ổn định, commit đồng thời RobotConfig h, main cpp, ESP32 source, giao diện và tài liệu protocol.
29. Gắn tag theo ngày và nội dung thử, lưu file hex hoặc bin kèm checksum và log chạy thật.
30. Mỗi thay đổi route phải cập nhật một bảng state gồm điều kiện vào, điều kiện ra, timeout, cảm biến chính và hành động lỗi.
31. Không tái sử dụng chân mà chưa cập nhật bảng chân và kiểm tra xung đột UART PWM encoder I2C SPI.
32. Mọi tham số tune từ giao diện phải được chép về RobotConfig h hoặc profile ESP32 rồi build lại. Chỉ ESP32 SAVE mới ghi NVS.
33. Giữ một test checklist theo map RED và BLUE, tốc độ thấp và tốc độ thi đấu, nguồn đầy và nguồn giảm, không tải và có tải.
34. Trước khi thi đấu, tạo bản release chỉ chứa state đang dùng, xóa hằng số không tham chiếu hoặc đánh dấu legacy bằng tên riêng.

Tóm tắt sử dụng tài liệu Khi robot có hành vi sai, bắt đầu từ state đang báo trên giao diện, đối chiếu bảng luồng AUTO, kiểm tra cảm biến quyết định của state đó, sau đó mới thay PID hoặc tốc độ. Nếu giao diện báo gửi thành công nhưng cơ cấu không chạy, kiểm tra lần lượt chiều gửi COM5, UART6, MECH STATUS, fault ESP32 và zero phần mềm.